

Project Documentation: GPS Tracking and Navigation System

This documentation covers the details of the main code for robot movement, including explanations of the code structure, key functions, methods, parameters, and mathematical concepts involved. Robot navigation is based on coordinates gathered by GPS, which is later sent to Firebase server to store and retrieve GPS data. The system also incorporates pathway navigation, compass movement, and pin location updates.

1. Project Structure Overview

The project is divided into four primary code files:

1. **GPS to Firebase (gpstofirebase)**
2. **Pin Location Updater (pinlocation updater)**
3. **Compass Movement**
4. **Pathway Navigation**

Each of these modules interacts with the hardware components, processes GPS data, and communicates with Firebase. The code utilizes methods to send GPS coordinates to Firebase, receive updated pin locations, and calculate routes for movement along a predefined path.

2. Code Flow and Explanation

2.1. GPS to Firebase (gpstofirebase)

This component is responsible for collecting GPS data and sending it to Firebase. The primary function here is to constantly monitor the GPS coordinates and transmit them over the internet using the ESP8266.

Key Code Elements:

- **GPS Module Interface:**
The GPS data is gathered using a GPS module connected to the ESP8266 via UART. Typically, the **TinyGPS++** library is used to parse GPS data such as latitude, longitude, and altitude.

- **Sending GPS Data to Firebase:**

The ESP8266 connects to Firebase using the Firebase Arduino library. The GPS data is sent using the `Firebase.setFloat()` function for both latitude and longitude.

cpp

CopyEdit

```
Firebase.setFloat(firebaseData, "/gps/latitude", latitude);  
Firebase.setFloat(firebaseData, "/gps/longitude", longitude);
```

- **Data Transmission:** Each time the GPS coordinates change, the new latitude and longitude are sent to Firebase under the paths `/gps/latitude` and `/gps/longitude`.

```
void sendGPSToFirebase(float latitude, float longitude) {  
    if (Firebase.setFloat(firebaseData, "/gps/latitude", latitude)) {  
        Serial.println("Latitude sent to Firebase");  
    }  
    if (Firebase.setFloat(firebaseData, "/gps/longitude", longitude)) {  
        Serial.println("Longitude sent to Firebase");  
    }  
}
```

2.2. Pin Location Updater (`pinlocation updater`)

This component listens for pin updates from Firebase, the pins are set by user through interface and they will determine the robots pathway.

Key Code Elements:

- **Firebase Data Retrieval:**

The code retrieves pin data from Firebase using `Firebase.getFloat()` to fetch updated latitude and longitude.

cpp

CopyEdit

```
float latitude = Firebase.getFloat(firebaseData,  
    "/pins/pin1/latitude");  
float longitude = Firebase.getFloat(firebaseData,  
    "/pins/pin1/longitude");
```

- **Data Handling:**

Once the data is retrieved, the system processes it and updates the pin location. The pins are stored in a structure (e.g., an array or a `Pin` class).

cpp

CopyEdit

```
struct Pin {
    String name;
    float latitude;
    float longitude;
};

Pin pin1;
pin1.latitude = latitude;
pin1.longitude = longitude;
pin1.name = "Pin 1";
```

- **Pin Location Update Function:**

When a new pin is received, the system updates the coordinates for that pin in the navigation system.

```
void updatePinLocation(String pinName, float latitude, float
longitude) {
    if (pinName == "Pin 1") {
        pin1.latitude = latitude;
        pin1.longitude = longitude;
    }
}
```

2.3. Compass Movement

This section uses the compass or IMU (Inertial Measurement Unit) to calculate the orientation (heading) of the system. The ESP8266 reads heading data from the compass, which helps in adjusting the device's path to follow the pre-defined waypoints.

Key Code Elements:

- **Compass Reading:**

The system communicates with the compass via I2C to fetch heading data.

```
int heading = compass.getHeading(); // Fetch the heading in degrees
```

- **Coordinate Calculation Using Heading:**

The system uses trigonometry (specifically sine and cosine) to update the position based on the current heading and movement distance. This involves using the current heading to adjust the device's position along the X and Y axes.

```
float deltaX = distance * sin(heading * DEG_TO_RAD);  
float deltaY = distance * cos(heading * DEG_TO_RAD);
```

- **Path Adjustment:**

Based on the heading, the system calculates how to move toward the next pin in the sequence and adjusts accordingly.

```
void moveToPin(float targetLatitude, float targetLongitude) {  
    float deltaLatitude = targetLatitude - currentLatitude;  
    float deltaLongitude = targetLongitude - currentLongitude;  
  
    // Calculate heading to the next pin  
    float heading = atan2(deltaLongitude, deltaLatitude) * RAD_TO_DEG;  
  
    // Update movement direction based on heading  
    adjustMovement(heading);  
}
```

2.4. Pathway Navigation

The navigation component uses the pin locations to guide the device along a path. It calculates the distance and heading to the next pin and adjusts the device's movement accordingly.

Key Code Elements:

- **Distance Calculation:**

The distance between the current position and the target pin is calculated using the Haversine formula or by simple Euclidean distance.

```
float distance = sqrt(pow(targetLatitude - currentLatitude, 2) +  
                      pow(targetLongitude - currentLongitude, 2)) *  
EARTH_RADIUS;
```

Where:

- `targetLatitude`, `targetLongitude` are the target GPS coordinates.
- `currentLatitude`, `currentLongitude` are the current coordinates.
- `EARTH_RADIUS` is the radius of the Earth (approx. 6371 km or 3959 miles).

Heading Calculation:

To calculate the heading (angle) to the next pin, the `atan2()` function is used.

```
float heading = atan2(targetLongitude - currentLongitude,  
targetLatitude - currentLatitude);  
heading = heading * DEG_TO_RAD; // Convert to degrees if needed
```

- **Navigation Loop:**
The navigation algorithm continuously fetches the next pin's location, calculates the heading and distance, and adjusts the movement toward that point.

```
void navigatePath() {  
    for (int i = 0; i < numPins; i++) {  
        moveToPin(pins[i].latitude, pins[i].longitude);  
    }  
}
```

3. Key Functions and Parameters

- **Latitude and Longitude:**
These are the GPS coordinates that represent the position of the device. The latitude specifies the north-south position, and the longitude specifies the east-west position.

- **Heading:**
The angle between the north direction and the direction of motion, calculated using a compass or IMU. It is used to adjust the device's movement along the path.
- **Distance:**
The spatial separation between two points, which can be calculated using the Haversine formula or Euclidean distance.
- **Pathway Calculation:**
The system uses mathematical functions like `atan2()`, `sin()`, and `cos()` to calculate angles and distances between the current position and the target location.

4. How to Manipulate the Code

1. **Change Pin Locations:**
Modify the Firebase database to update the coordinates of the pins. The system will automatically read and process the updated coordinates.
2. **Adjust Movement Parameters:**
In the `moveToPin()` function, you can modify how the system adjusts its movement by changing the calculation of `deltaX`, `deltaY`, and the speed of movement.
3. **Update Pathway:**
You can modify the `navigatePath()` function to change the order of pins or add new pins to the sequence.
4. **Modify Heading Calculation:**
The `atan2()` function can be adjusted if you wish to implement more advanced movement algorithms such as obstacle avoidance or dynamic heading adjustments based on real-time feedback.

5. Mathematical Concepts

- **Haversine Formula:**
Used to calculate the great-circle distance between two points on a sphere, which is necessary for GPS distance calculations.
- **Trigonometry:**
Used for calculating the delta in X and Y positions based on the heading of the device.
- **Angle Calculation with `atan2()`:**
The `atan2()` function computes the angle from the horizontal axis to the line between two points, which is crucial for heading calculation.

6. Conclusion

This GPS-based tracking and navigation system, with integration to Firebase, provides a solid foundation for building autonomous systems. The code explains how to read GPS data, update

locations, and navigate using heading and distance calculations. The use of trigonometry and distance formulas allows for accurate pathfinding and movement toward predefined points.